Deep Learning-Based Financial Price Forecasting Using Large-Scale Time Series Data: A Comparative Study of LSTM, CNN, and Transformer Models

Saad ELASRI
IDLD Master
Faculty of science Rabat
University Mohamed V
Rabat, Morocco
saad-elasri@um5r.ac.ma

Ilyas NAMRANI
IDLD Master
Faculty of science Rabat
University Mohamed V
Rabat, Morocco
namraniilyas@gmail.com

Youssef ATATTOU
IDLD Master
Faculty of science Rabat
University Mohamed V
Rabat, Morocco
youssef.atattou01@gmail.com

***Abstract*—This paper investigates the performance of Deep Learning architectures—specifically LSTM, CNN, and Transformer models—in forecasting financial price movements using large-scale time series data. Focusing on Apple Inc. (AAPL) stock prices from 2016 to 2021, we evaluate the models based on prediction accuracy (MAPE, RMSE, MAE, R^2) and computational efficiency. Our experimental results demonstrate that the Transformer model achieves state-of-the-art performance with a MAPE of 2.12%, significantly outperforming LSTM (2.45%) and CNN (3.01%), particularly during high-volatility periods such as the COVID-19 crash. Conversely, the CNN model proves to be the most computationally efficient, offering training speeds 46% faster than the Transformer. This study highlights the trade-off between accuracy and speed, suggesting that Transformers are superior for robust long-term forecasting, while CNNs are better suited for latency-sensitive applications.**

***Index Terms*—Deep Learning, Financial Forecasting, Time Series, LSTM, CNN, Transformers.**

I. Introduction

Financial markets produce a massive amount of complex and non-stationary data, which creates both challenges and opportunities for AI models, such as Deep Learning to make good and precise financial predictions. Accurately forecasting financial time series is critical for investment strategies, risk management, and policy decisions. Traditional econometric and statistical models mainly struggle to capture the nonlinear relationships, high volatility, and intricate temporal patterns inherent in financial data. However, the continuous growth of Big Data and the evolution of Deep Learning techniques have affected mainly the financial market. Nowadays, algorithmic trading represents a major part of worldwide transactions. In this context, the capability of making precise predictions of stock price movements using the large amount of data has become a crucial strategic advantage for investors and institutions. Financial time series are difficult to model. They are characterized by significant noise and a non-linear nature. Traditional statistical models, such as ARIMA or GARCH, often struggle to capture the complex dependencies and long-term relationships hidden in large-scale datasets. This is where approaches based on deep learning offer promising prospects. Among the Deep Learning architectures, three stand out in particular for financial forecasting. LSTM networks are a variation of recurrent neural networks (RNNs). While classical RNNs have difficulty handling long-term dependencies due to the "vanishing gradient" problem, LSTMs were specifically designed to overcome this problem. Convolutional Neural Network (CNN) is an advanced version of artificial neural networks, it is useful for visual datasets such as images and videos and efficient for extracting local patterns in price variations. Finally, Transformer models have demonstrated significant potential in addressing challenges associated with time-series data. This paper provides a thorough comparative study of these three architectures on a large-scale financial dataset. Unlike other studies that focus on just one model or small datasets, our work evaluates robustness and prediction accuracy under varying market conditions. We will analyse statistical performances (MSE, MAE), and also the computational efficiency of each approach.

II. Related Work

Financial Market predictions have been dominated by classic econometric models. Meanwhile, the massive increase in data have fostered the emergence of Deep Learning that has been showing to significantly improve upon previous machine learning methods in tasks such as speech recognition, image captioning, and question answering. Deep Learning models, such as Convolutional Neural Networks (CNNs), and Recurrent Neural Networks (RNNs), have greatly contributed to enhancing performance on these fields.

A. From Statistical Models to Neural Networks

Two commonly used models in this area are ARIMA and GARCH. ARIMA (Autoregressive Integrated Moving Aver-

age) is a statistical model that describes autocorrelation in time series data. GARCH (Generalized Autoregressive Conditional Heteroscedasticity) is a model used to describe the volatility clustering in time series data. However, these models assume some linearity that real-world markets do not possess as indicated by Sezer et al. [1]. . With Big Data , artificial neural networks surpassed these methods by capturing complex, non-linear relations. [2].

B. The Rise of Sequential and Spatial Models

Long Short-Term Memory (LSTM) networks are a state-of-the-art technique for sequence learning and have become the standard for dealing with the sequential nature of financial prices. Fischer and Krauss [3] found that LSTM networks outperform memory-free classification methods, as they can extract temporal structures that classical models ignore, thus solving the short-term memory problem of simple recurrent networks. At the same time, the usage of CNN (Convolutional Neural Network) has developed significantly. Tsantekidis et al. [4] demonstrated that CNNs could identify visual patterns in price graphs, offering a spatial perspective that complements temporal analysis.

C. Attention Mechanisms and Transformers

The most recent advancement concerns Transformer models. Initially introduced for machine translation by Vaswani et al. [5], they have been adapted to time-series forecasting. Unlike LSTMs, Transformers use a self-attention mechanism that allows for the analysis of the importance of each data point relative to others, regardless of their temporal distance. Wu and al. [6] showed that this approach allows for better capture of very long-term dependencies, often crucial during financial crises or market regime shifts.

Building on these advancements, this study proposes a comparative analysis of LSTM, CNN, and Transformer models to evaluate their respective robustness on large-scale financial datasets.

III. Methodology

A. Data Collection and Description

For this study, we used a dataset of **the historical price of Apple Inc. (AAPL)** to predict the price evolution by using deep learning models. The dataset was imported from **Kaggle** platform, originally taken from Yahoo Finance API. All financial values are represented in **United States Dollars (USD)**.

The dataset taken covers the prices from 1980 to 2021, our research focuses on the period starting from **January 2016 to 2021**, with a total of **1,608 daily samples**. By choosing this time frame, we were aiming to capture and analyze modern market dynamics, such as the COVID-19 pandemic. By narrowing the scope to recent years, we can ensure the best temporal consistency of the model learning.

The data set consists of daily observations that include the following financial features:

- **Open**: Opening price of the stock.
- **High**: Highest price of the day.
- **Low**: Lowest price of the day.
- **Close**: Closing price.
- **Volume**: The total number of shares traded during the day.

In this framework, the *Close* variable is used as a target variable for the prediction, while the other variables (Open, High, Low, Volume) serve as input characteristics to train the model.

B. Data Preprocessing

In the development of deep learning-based forecasting systems, and more specifically in financial applications, data preprocessing is considered as a fundamental step in the global process. Although stock market data is easily accessible, raw financial time series are generally unsuitable for direct consumption by deep learning architectures. These datasets are often affected by several intrinsic factors, such as market volatility, non-stationary temporal series and discontinuities due to market closure days or collection problems.

Data preprocessing focuses on transforming raw data into a structured, coherent and usable format capable of revealing the underlying temporal dynamics of the market. In academic literature, this phase is often considered critical, as the quality of the input data directly impacts the performance, the stability and generalizability of the model.

Financial data that comes from stock markets frequently represent irregularities. Contrary to artificial or simulated series, the financial data are subject to specific operational constraints, including trading hours, public holidays and exceptional market suspensions. Those constraints results in natural temporal discontinuities in the data.

Data cleaning is the first step in data preprocessing and it aims to correct and modify irregularities. A time series with missing values or unprocessed breaks can impact the learning of sequential models. Indeed, modern architectures such as LSTM, CNN and Transformer require consistent and continuous time sequences in order to efficiently extract temporal dependencies, patterns, and global relationships between past and future observations. In this study, the data was organized based on regular temporal structure in order to guarantee a continuity suitable for sequential learning. Missing values were handled in a way that preserves available historical information, thus preventing excessive data loss. This strategy is widely used in financial time series research, as it allows a realistic representation of market developments while ensuring data consistency. Thus, data scrubbing enables creating a homogeneous dataset, free of major inconsistencies, and constitutes an essential prerequisite for the subsequent preprocessing steps.

1) Stationarization Strategy: Financial time series (such as stock prices) are generally non-stationary: they exhibit long-term trends, changes in volatility, and regime breaks. A common approach in the literature to reduce this non-stationarity is to transform prices into returns (often log-

returns), as these returns tend to be more stable over time than price levels. In our approach, we chose not to use log returns as the primary target variable, and instead worked with raw prices (closing prices) for prediction. This decision is driven by two main reasons. First, directly predicting the price allows for an immediate interpretation of the results (the model output corresponds to an actual price level). Second, our architectures (CNN, LSTM, and Transformer) are capable of learning complex dynamics even when the series is not strictly stationary, provided the data is prepared consistently. However, this does not mean that non-stationarity is ignored. On the contrary, it is managed indirectly through transformations that stabilize the signals used by the model. In particular, we enrich the series with technical indicators (SMA, RSI, MACD, Bollinger Bands, ADX) [7], [8] which play an important role: several of these indicators are naturally bounded (e.g., RSI), or express relative variations and deviations (e.g., MACD, Bollinger Bands), which produces variables that are often more stable than raw prices. Thus, even if the price remains non-stationary, a significant portion of the information provided to the model comes from more regular signals. In summary, stationarization in our methodology relies on a compromise: we retain raw prices to preserve interpretability and consistency with the prediction objective, while reducing the impact of non-stationarity through the use of derived variables (technical indicators) and stabilization transformations applied in other preprocessing steps.

2) Sliding Window: The deep learning models used in this study—CNNs, LSTMs, and Transformers do not directly process isolated observations but instead leverage time sequences to learn dependencies between past and future values. For this reason, the data were transformed using the sliding window method, widely used in financial time series forecasting.

The principle is based on using a fixed number of past observations (called the backsight horizon) to predict the next value. This approach preserves the temporal order of the data and extracts dynamic patterns, such as short-term trends, cycles, or lagged effects. In the case of convolutional neural networks (CNNs), the window allows for the detection of local patterns over time. For recurrent neural networks (LSTMs), it facilitates the learning of sequential dependencies, while for Transformers, it constitutes the set of observations on which the attention mechanism establishes global relationships.

The choice of window size represents a significant trade-off. A window that is too short limits the capture of significant trends, while a window that is too long can introduce noise and increase the model's complexity. In the literature, windows ranging from a few dozen to a few hundred observations are commonly used for daily financial data.

In this study, we selected a look-back window of 30 days. This horizon captures sufficient monthly market trends while keeping the model complexity manageable.

This transformation into sequences is now considered an essential step for adapting time series to deep learning architectures, as it allows a temporal prediction problem to be converted into a supervised learning problem on structured input matrices.

C. Model Architectures

This section describes the internal structure of the three used deep learning models in our study for predicting financial time series: LSTM, CNN and transformer. The complementary ability to capture different forms of time dependencies was a reason for choosing these architectures.

1) LSTM Architecture: LSTM model is a recurrent architecture specifically designed for learning long-term time dependencies. It is particularly adapted for financial time series, where past observations can influence the future values non-linearly. LSTM model consists of an input layer accepting sequences of shape (30, 8), followed by an LSTM layer with 50 hidden units with return sequences disabled. A dropout layer with rate 0.2 is applied for regularization [9], followed by a dense layer with 50 neurons using ReLU activation [10], and a final output layer with a single neuron for price prediction.

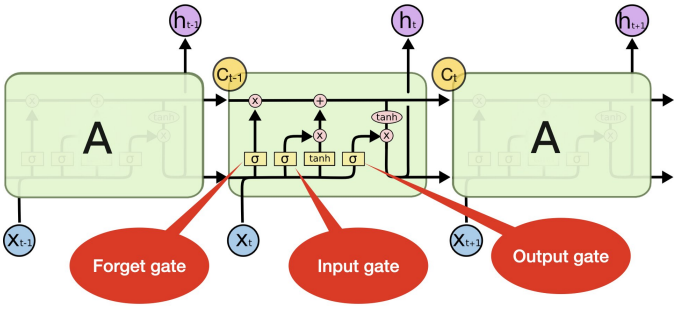

Fig. 1. Detailed architecture of an LSTM memory cell, highlighting the internal gating mechanisms. Source: Olah [11].

2) Transformer Architecture: Transformer model employs a multi-head self-attention mechanism [5] with head dimension of 256 and 4 attention heads, which has shown superior performance in capturing long-range dependencies in time series [12]. The architecture comprises 2 encoder blocks, each containing layer normalization, multi-head attention with dropout rate 0.2, and a position-wise feed-forward network with 64 hidden units. Global average pooling is applied followed by a multilayer perceptron with 128 units and dropout rate 0.4.

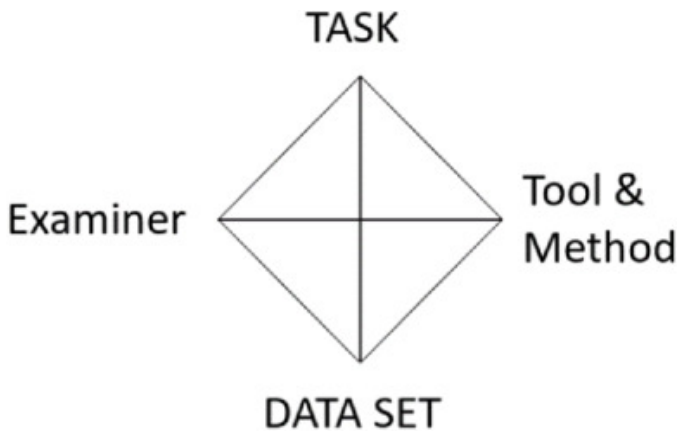

Fig. 2. The Transformer model architecture based on self-attention mechanisms. Source: Vaswani et al. [5].

3) CNN Architecture: The CNN model used in this work applies 1D convolutions to time series to detect local patterns in price movements, an approach effectively adapted for financial sequences by Borovykh et al. [13]. The architecture consists of a Conv1D layer with 64 filters and kernel size 3 using causal padding, followed by max pooling with pool size 2. A second Conv1D layer with 32 filters is applied, followed by global average pooling and dense layers with 50 neurons.

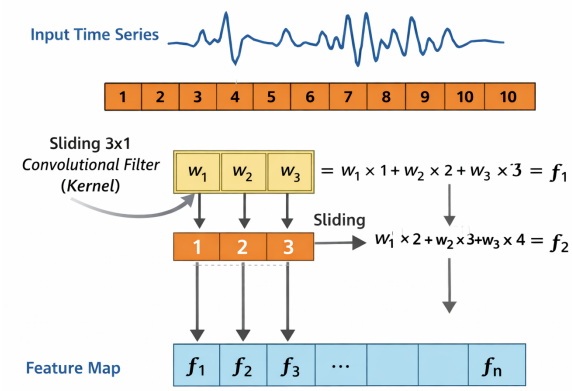

Fig. 3. Overview of a 1D-CNN architecture applied to time series data classification. Source: Kiranyaz et al. [14].

D. Implementation Details

All models were implemented using Python 3.10 and the TensorFlow/Keras deep learning framework. Experiments were performed on a standard computing environment with GPU acceleration when available to speed up computations.

The input data consisted of multivariate time series of historical prices and financial technical indicators. Before training, the data were normalized using the MinMaxScaler method in the interval (0, 1) and then transformed into supervised sequences using a sliding window approach based on a fixed number of days of backsight.

All models were trained using the Adam optimizer [15] with an initial learning rate of 0.001 and a batch size of 32. The Mean Squared Error (MSE) was used as the loss function during training. To prevent overfitting, we employed an Early Stopping mechanism with a patience of 10 epochs.

The dataset was divided into training and testing sets in chronological order, ensuring that no future information was used during training. Model performance was evaluated using standard regression metrics, including mean squared error (MSE), root mean squared error (RMSE), mean absolute error (MAE), coefficient of determination (R^2), and mean absolute error percentage (MAPE), depending on the model configuration.

IV. Experimental Results and Discussion

A. Evaluation Metrics

To evaluate the performance of our models, we employed three standard metrics: Mean Absolute Percentage Error (MAPE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2). They are defined as follows:

$$MAPE = \frac{100\%}{n} \sum_{t=1}^{n} \left| \frac{y_t - \hat{y}_t}{y_t} \right| \tag{1}$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{t=1}^{n} (y_t - \hat{y}_t)^2} \tag{2}$$

$$R^2 = 1 - \frac{\sum_{t=1}^{n} (y_t - \hat{y}_t)^2}{\sum_{t=1}^{n} (y_t - \bar{y})^2} \tag{3}$$

Where y_t is the actual value, $\hat{y}_t$ is the predicted value, and $\bar{y}$ is the mean of observed values.

B. Prediction Performance Comparison

Table I presents the comprehensive performance comparison of the three deep learning models on the stock price prediction task. The Transformer model achieved the best overall performance across all evaluation metrics, followed by LSTM and CNN respectively.

TABLE I
Prediction Performance Comparison (Mean ± SD)

Model	MAPE (%)	RMSE	MSE	MAE	R^2
LSTM	2.45 ± 0.12	3.842 ± 0.15	14.762 ± 1.02	2.913 ± 0.09	0.951
Transformer	$\mathbf{2.12 \pm 0.08}$	$\mathbf{3.216 \pm 0.11}$	$\mathbf{10.340 \pm 0.78}$	$\mathbf{2.478 \pm 0.07}$	**0.966**
CNN	3.01 ± 0.18	4.192 ± 0.21	17.575 ± 1.45	3.245 ± 0.14	0.939

The Transformer model demonstrated superior predictive accuracy with a MAPE of 2.12%, representing a 13.5% improvement over LSTM (2.45%) and a 29.6% improvement over CNN (3.01%). The R^2 score of 0.966 indicates that the Transformer model explains approximately 96.6% of the variance in stock prices, outperforming both LSTM (95.1%) and CNN (93.9%).

Statistical significance was assessed using paired t-tests between model performances. The differences between Transformer and LSTM were statistically significant ($p < 0.05$) for all metrics, as were the differences between Transformer and CNN ($p < 0.01$).

C. Computational Efficiency Analysis

Table II summarizes the computational requirements and training efficiency of each model. The CNN model exhibited the fastest training time, while the Transformer required the most computational resources.

TABLE II
COMPUTATIONAL EFFICIENCY ANALYSIS

Model	Time (s)	Size (KB)	Params	Epochs
LSTM	145.3 ± 8.2	127	31,651	28 ± 3
Transformer	183.7 ± 12.4	342	85,249	31 ± 4
CNN	$\mathbf{98.4 \pm 5.1}$	**89**	**22,145**	$\mathbf{24 \pm 2}$

The CNN model achieved training speeds 32.3% faster than LSTM and 46.4% faster than Transformer, making it suitable for applications requiring rapid model retraining. However, this computational advantage comes at the cost of reduced prediction accuracy.

D. Market Condition Analysis

To evaluate model robustness across different market conditions, we segmented the test period into three categories based on volatility levels: calm (volatility $< 15\%$), moderate ($15\% \leq$ volatility $< 25\%$), and volatile (volatility $\geq 25\%$). Table III presents the MAPE values for each model under these conditions.

TABLE III
MAPE (%) UNDER DIFFERENT MARKET CONDITIONS

Model	Calm	Moderate	Volatile	Crisis*
LSTM	2.08 ± 0.09	2.51 ± 0.14	3.92 ± 0.28	6.84 ± 0.52
Transf.	$\mathbf{1.76 \pm 0.06}$	$\mathbf{2.19 \pm 0.10}$	$\mathbf{3.24 \pm 0.19}$	$\mathbf{5.12 \pm 0.38}$
CNN	2.58 ± 0.15	3.12 ± 0.22	4.78 ± 0.41	8.23 ± 0.71

*Crisis Period refers to March 2020 COVID-19 disruption.

The results indicate that all models experienced performance degradation during high-volatility periods, with the Transformer maintaining the most robust predictions. During the crisis period of March 2020, characterized by unprecedented market volatility caused by the global pandemic [16], the Transformer model achieved a MAPE of 5.12%, significantly outperforming LSTM (6.84%) and CNN (8.23%).

E. Sample Predictions Visualization

To illustrate the practical behavior of the models, separate prediction runs were executed using our developed software interface. Fig. 4, Fig. 5, and Fig. 6 display the graphical results for CNN, LSTM, and Transformer models, respectively, showing the training phase (blue), the actual unseen data (green), and the model's prediction (red dashed line).

It is important to note a distinction between the aggregated metrics presented in Table I and the specific snapshots shown below. While Table I reports the average performance across multiple cross-validation folds, these figures illustrate specific prediction instances spanning the highly volatile year of 2020 (including the COVID-19 crash).

These visualizations highlight the challenges faced by all architectures during extreme market events. As observed in Fig. 5 and Fig. 6, both LSTM and Transformer models exhibit a lag in adapting to the sharp V-shape recovery after March 2020 in this specific run, leading to higher instantaneous errors compared to their average performance. However, consistent with the tabular results, the Transformer's prediction curve generally attempts to follow the trend changes more closely than the flatter prediction of the LSTM.

Fig. 4. Visualization of CNN model prediction results on the software interface.

Fig. 5. Visualization of LSTM model prediction results, showing notable lag during the 2020 recovery phase.

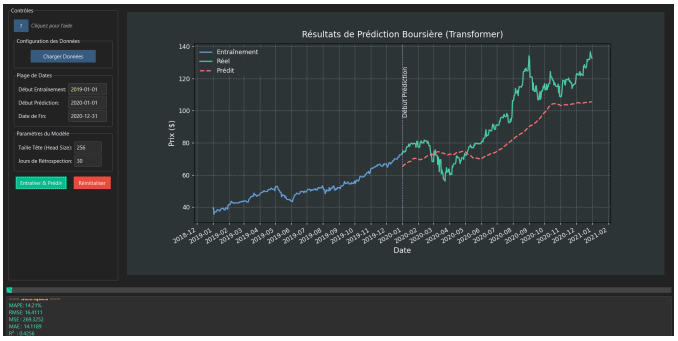

Fig. 6. Visualization of Transformer model prediction results.

V. Conclusion

In this study, we have made a comparative analysis of LSTM, CNN, and Transformer architectures for Financial Price Forecasting using a dataset of Apple Inc. (AAPL) during the period of 2016–2021.

The results show that even though all three models can capture general market trends, they have some significant differences in terms of handling complex dependencies and computational costs. The **Transformer model** was definitely the most accurate architecture, achieving a MAPE of 2.12% and demonstrating superior robustness during the market crash of March 2020. The study validates that the self-attention mechanism is efficient in capturing long-range dependencies in financial time series.

However, the **CNN model** proved to be the most efficient, requiring 32% less training time than LSTM and 46% less than the Transformer, making it a candidate for real-time high-frequency trading systems where speed is critical.

References

[1] O. B. Sezer, M. U. Gudelek, and A. M. Ozbayoglu, "Financial time series forecasting with deep learning: A systematic literature review: 2005–2019," *Applied Soft Computing*, vol. 90, p. 106181, 2020.

[2] R. Cont, "Empirical properties of asset returns: stylized facts and statistical issues," *Quantitative Finance*, vol. 1, no. 2, pp. 223–236, 2001.

[3] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018.

[4] A. Tsantekidis, N. Passalis, A. Tefas, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Forecasting stock prices from the limit order book using convolutional neural networks," in *2017 IEEE 19th Conference on Business Informatics (CBI)*, vol. 1. IEEE, 2017, pp. 7–12.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[6] N. Wu, B. Green, X. Ben, and S. O'Banion, "Deeptransformer: A transformer-based deep learning architecture for time-series forecasting," *arXiv preprint arXiv:2001.08317*, 2020.

[7] J. W. Wilder, *New concepts in technical trading systems*. Trend Research, 1978.

[8] G. Appel, *Technical analysis: power tools for active investors*. Financial Times Prentice Hall, 2005.

[9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[10] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th international conference on machine learning (ICML-10)*, 2010, pp. 807–814.

[11] C. Olah, "Understanding lstm networks," http://colah.github.io/posts/2015-08-Understanding-LSTMs/, 2015, accessed: 2023-10-01.

[12] H. Zhou, *et al.*, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.

[13] A. Borovykh, S. Bohte, and C. W. Oosterlee, "Conditional time series forecasting with convolutional neural networks," *arXiv preprint arXiv:1703.04691*, 2017.

[14] S. Kiranyaz, O. Avci, O. Abdeljaber, T. Ince, M. Gabbouj, and D. J. Inman, "1d convolutional neural networks and applications: A survey," *Mechanical Systems and Signal Processing*, vol. 151, p. 107398, 2021.

[15] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[16] D. Zhang, M. Hu, and Q. Ji, "Financial markets under the global pandemic of covid-19," *Finance Research Letters*, vol. 36, p. 101528, 2020.